PARSHWANATH CHARITABLE TRUST'S
A.P. SHAH INSTITUTE OF TECHNOLOGY
Department of Computer Science and Engineering
Data Science

SCIAM-Smart Collaborative IDE with AI Support and Session Management

Group No. 8

Student ID	Group member
22107023	Shuban Devanpelli
22107002	Rishi Bije
22107056	Dhanraj Bacche
22107055	Yash Kamble

Project Guide
Ms. Sarala Mary
Ms. Richa Singh

Contents

- Abstract
- Introduction
- Objectives
- Literature Review
- Research Gap
- Problem Definition
- Scope
- Technological Stack
- Proposed System Architecture/Working/Module Diagram
- Prototype Design Demonstration
- Implementation Status
- Review Suggestions (Given in Last meeting)

Abstract

SCIAM is a web-based Smart Collaborative IDE integrating real-time coding, AI assistance, and intelligent contribution tracking. It addresses challenges of distributed teamwork, inefficient communication, and lack of AI support in existing tools. Features include synchronized editing with host permissions, embedded chat, instant AI-driven debugging, and performance metrics. As a single-file, browser-based solution, SCIAM boosts efficiency, accountability, and transparency in team-based development.

Sustainable Development Goals(SDG) mapped

SDG mapped to project: 4

SDG 4: Quality Education
Mapping: The collaborative IDE promotes peer-to-peer learning and skill development by allowing multiple students or developers to work on the same codebase in real time. The AI assistant further supports learning by offering guidance, debugging help, and personalized feedback.

Introduction

- SCIAM is a web-based IDE that allows 4–5 members to code together in a host-controlled session with organized teamwork.
- An integrated chat system ensures smooth interaction, removing the need for external messaging tools.
- Provides instant code correction, debugging suggestions, and tracks individual contributions for transparency.
- Maintains a single downloadable file for each session, making collaboration easy

Motivation

- Existing IDEs lack integrated real-time collaboration, forcing teams to rely on multiple external tools. Developers face inefficiency due to context switching, poor communication, and limited AI-driven coding support.
- SCIAM is motivated by the need for a unified platform that simplifies teamwork, boosts productivity, and ensures transparent contribution tracking.

Objectives

1. To develop a web-based collaborative IDE that allows 4–5 members to code in real time with host-controlled write permissions.
2. To enable built-in chat functionality for seamless communication between team members during sessions.
3. To create a custom AI module for real-time code correction, suggestions, and debugging support.
4. To implement an AI-powered tracking system to monitor and report individual code contributions to the host.

Literature Review

Sr.no	Title	Author(s)	Year	Methodology	Drawback
1	A Proposal of the Gamification of Programming Learning Assistant System[1]	Htoo Htoo Sandi Kyaw, Keiichi Kaneko, Nobuo Funabiki	2024	Proposal and prototype design of gamifying a Programming Learning Assistant System (PLAS) using game mechanics to improve programming comprehension and engagement for novice students. The system incorporates various programming exercises and a game environment implemented in Unity.	The paper presents a proposal and initial prototype; lacks extensive evaluation data and empirical validation of effectiveness.
2	CodeQuest - A Comprehensive Code Learning and Collaboration Platform[2]	Vaishali Savale, Vedashri Chaudhari, Tejaswini Wanare, Balamurali Nambiar, Manthan Vasant, Sanskruti Umak	2024	Development of an integrated DSA learning platform with features such as chatbot powered by Llama 2-7B, real-time code sync, code editor, and code fusion for live feedback; built using React, Socket.IO, Express, and AMKC API for code execution.	The paper highlights the drawbacks of existing platforms including lack of full integration of personalized tutoring, real-time collaboration, and instant feedback; also, chatbot systems may fail or have limited dynamic behavior; current platform may still improve with advanced AI, language support, gamification, and mobile app.
3	Towards Conflict-Free Collaborative Modelling Using VS Code Extensions[3]	Rijul Saini, Gunter Mussbacher	2021	Development of a framework extension (tColab) for conflict-free real-time collaborative modelling in textual goal-oriented requirement language (TGRL). Proposed VS Code extension for TGRL with two conflict-resolution solutions: (1) VS Code Live Share extension for real-time collaboration with conflict management, cursor awareness, and undo handling; (2) VS Code Teletype extension (work in progress) using CRDTs for decentralized conflict-free syncing. Evaluation through	Current reliance on VS Code Live Share extension cannot be used on non-VS Code platforms like Eclipse Che and Theia due to licensing; Teletype solution is still work in progress; handling modeling conflicts beyond text (semantic conflicts) requires improved awareness; initial scope limited to TGRL, extending to full URN still needed.

Literature Review

Sr.no	Title	Author(s)	Year	Methodology	Drawback
4	Annotation-based Semantic Conflict Prevention in Real-Time Collaborative Programming: Approach, Techniques, Prototype, and User Study[4]	Mingjie Wang, Bicheng Fang, Jinfeng Jiang, Hongfei Fan	2024	Developed an annotation-based semantic conflict prevention approach with flexible manual and dynamic/static locks in code regions for real-time collaborative programming. Implemented in CoIDEA prototype (IntelliJ IDEA plugin). Conducted early user study (questionnaires, interviews), design objectives, algorithm development, user and performance evaluations demonstrating usability, efficiency, and large-scale support.	Potential overhead on user for manually applying locks; initial prototype limited to IntelliJ platform; collaborative awareness features can be further improved; balancing flexibility with ease of use remains a challenge
5	Collaborative Code Editors - Enabling Real-Time Multi-User Coding and Knowledge Sharing[5]	Khushwant Virdi, Anup Lal Yadav, Azhar Ashraf Gadoo, Navjot Singh Talwandi	2023	Conducted a comprehensive literature review of collaborative code editors and their evolution; analyzed key technologies like WebSocket communication and Operational Transformation (OT) algorithms; designed and developed a prototype collaborative code editor using NodeJS, React, and Express.js; implemented OT for conflict resolution; evaluated the prototype through performance metrics (latency, synchronization efficiency) and user experience feedback; conducted comparative analysis with existing platforms.	Challenges persist with latency, synchronization, and conflict resolution complexity; real-time collaboration still faces technical and logistical issues; prototype improvements needed for scaling and ease of use.

Sr.no	Title	Author(s)	Year	Methodology	Drawback
6	Building Temporary Isolated Workspace in Real-Time Collaborative Programming Environment[6]	Jinfeng Jiang, Yuxiang Xie, Bicheng Fang, Mingjie Wang, Hongfei Fan	2023	Proposed a novel approach called RECON (Reversion of Error-free Code with Workspace Isolation) to support compilation and debugging during real-time collaborative programming. Developed techniques to maintain error-free code copies alongside active editing code, and create temporary isolated workspace for error-free compilation/debugging. Implemented prototype in CoIDEA plugin for IntelliJ IDEA. Evaluated via experimental user interface scenarios and system performance.	Current system triggers syntactic error detection on every user operation, causing potential resource overhead; editing is not allowed in silent (isolated) mode; further work needed for optimizing resource usage and enabling editing during debugging sessions.
7	CodeXchange: Leaping into the Future of AI-Powered Code Editing[7]	Mihir Agrawal, Jatin Goyal, Mradul Goyal, Pratham Sukhija, Jaya Sharma, D. Franklin Vinod	2024	Developed “CodeXchange,” an online AI-powered code editor that supports neural code translation across Python, Java, and C++, GPT-3–based comment generation, ML-driven code optimization, and real-time collaborative editing via WebSockets with operational transformation; completed literature review and architecture design, built a working prototype, and evaluated it through user feedback and performance analysis.	Challenges include high resource and computational cost for AI model training and inference, complexity in operational transformation for conflict resolution especially in large shared codebases, security concerns for data/code sharing, scalability for heavy user traffic, and integration challenges with third-party tools.

Sr.no	Title	Author(s)	Year	Methodology	Drawback
8	AI Based Code Error Explainer using Gemini Model[8]	Ankita Amburle, Cheryl Almeida, Oswin Lopes, Nathan Lopes	2024	Built an AI tool for real-time error detection, code explanations, and tailored fixes across Python, C, and Java, using language parsers, command execution, a Bard-based explainer, and live execution with error capture—delivered with color-coded, syntax-highlighted output and validated through user testing showing high accuracy and positive feedback.	Currently limited language support (Python, C, Java); requires integration of more languages for broader applicability; potential need for ongoing refinement in AI explanation accuracy and interactive features.
9	AI Coding: Learning to Construct Error Correction Codes[9]	Lingchen Huang, Huazi Zhang, Rong Li, Yiqun Ge, Jun Wang	2020	Proposed an AI-driven ECC design using a constructor–evaluator loop: reinforcement learning (policy gradient, A2C) and genetic algorithms generate and refine linear block and polar code constructions, while a simulator evaluates decoding performance across channel conditions; results show the learned codes match or sometimes surpass classical designs under comparable settings.	High computational cost for training AI models; complexity of reinforcement learning and genetic algorithms; code evaluator accuracy and efficiency impact overall design; current work focuses on offline training, limiting adaptability to real-time changes.

Sr.no	Title	Author(s)	Year	Methodology	Drawback
10	An Empirical Study on the Usage of Transformer Models for Code Completion[10]	Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, Gabriele Bavota	2021	An Empirical Study on the Usage of Transformer performed a thorough empirical study on the effectiveness of Transformer models like RoBERTa and T5 for code completion tasks. Using large Java and Android datasets, they assessed model performance at various code granularities (token, construct, block) and compared results with traditional n-gram baselines. The study found T5 achieved the best results overall, but all models showed sharp accuracy drops as the code blocks to be predicted grew larger. Additionally, the research was limited to Java/Android and required substantial computational resources for model training and evaluation, highlighting the need for broader language studies and practical developer assessments.	Currently limited to a prototype stage; potential challenges in scaling engagement for diverse learner populations; effectiveness dependent on game design quality and exercise diversity; no detailed evaluation results presented.
11	Code Generation Using Self-interactive Assistant[11]	Zixiao Zhao, Jing Sun, Cheng-Hao Cai, Zhiyuan Wei	2024	Developed "SelfCoder," a self-interactive code generation framework using the OpenAI Assistant API and GPT-4. Combines Knowledge Retrieval and Code Interpreter tools to iteratively generate, test, and optimize code based on user instructions. Evaluated on HumanEval and MBPP datasets with comparative analysis, ablation studies, and user testing to demonstrate improved performance and reliability.	Limitations include struggles with ambiguous inputs, inconsistent stability due to fixed API parameters, high operational costs, and constrained knowledge base size limiting domain-specific effectiveness.

Sr.no	Title	Author(s)	Year	Methodology	Drawback
12	The Development of a Collaborative Learning Tool for AI Education: Corobolab[12]	Haibin Lai, Peng Peng, Yuhui Lv, Peiwei Cai, Qiongxiong Ma, Zhun Zhang	2025	Developed "Corobolab," a collaborative learning tool integrating software and hardware platforms for K-12 AI education. The system supports real-time collaborative programming with physical robot kits, sensors, and cameras. Employed multi-terminal graphical programming, modular hardware interfaces, and a networked feedback loop between software and hardware. Evaluated via pilot applications in primary and middle schools, with qualitative user feedback and observations.	Limited user interface clarity for hardware setup; network issues and system responsiveness impact experience; requires user training and technical support; currently targeted at K-12, needing adaptation for advanced curricula.
13	LiteDoc: Make Collaborative Editing Fast, Scalable, and Robust[13]	Saptaparni Kumar, Haochen Pan, Roger Wang, Lewis Tseng	2020	Proposed LiteDoc, a collaborative editor with simplified, provably correct semantics: documents are split into sections modeled as single-writer, multi-reader atomic registers, with writer-locks to prevent conflicts. Simulations against consensus-based and server-ordered editors show higher throughput, lower latency, and better scalability and robustness under load.	Limited feature set compared to general editors (e.g., Google Docs); restricted concurrency by allowing only one writer per section at a time; potentially less intuitive for some users used to simultaneous edits; initial implementation and UI still in development.

Sr.no	Title	Author(s)	Year	Methodology	Drawback
14	AI-Driven Web Application Automated Testing and Code Optimization[14]	Junbang Ma	2025	Developed a deep learning–based testing framework that combines LSTM-driven automated test generation, Q-learning for adaptive test execution, and CNN-based code quality evaluation, improving efficiency and optimization for large web applications; validated on diverse real-world projects with higher coverage, accuracy, and system performance.	High computational and operational resource demands; limitations in programming language support; requirement for extensive data preprocessing; potential challenges in adapting to rapidly changing codebases.
15	Multiple Authors Identification from Source Code Using Deep Learning Model[15]	Abdul Mannan Omi, Monir Hossain, Md Nahidul Islam, Tanni Mittra	2021	Proposed a method for identifying multiple authors in source code by segmenting code and analyzing coding style using deep learning models. Utilized Abstract Syntax Tree (AST) and code2seq to extract structural features. Employed DNN, SVM, and LSTM classifiers to identify authorship segments and then combined results for multi-author detection. Dataset comprised 3021 Java files from 40 authors; achieved up to 96.7% accuracy with DNN.	Limited to Java language in current study; potential scalability challenges for larger datasets; dependency on AST parsing quality; needs extension to multiple programming languages for broader applicability.

Research Gap(Limitations of existing systems)

- **Self-interactive loops** — Higher accuracy, but added latency and cost; best for high-stakes fixes, not quick autocompletes.
- **Co-editing controls** — Need semantic locks/CRDTs to avoid conflicts; section writer-locks give deterministic merges at scale.
- **Shadow snapshots** — Keep a parallel clean build to compile/test privately; avoids blocking teammates during active edits.
- **Tiny re-rankers** — Great speed/size for on-device IDEs; limited context window vs. large models, so misses long-range deps.
- **Seq2seq Transformers** — Better at block-level completion; use confidence thresholds and fallback to reduce hallucinations.
- **Classroom AI + collab** — Boosts engagement and personalization; requires monitoring, age-appropriate content, and audit logs.

Problem Definition

Modern software development teams struggle with collaborative coding due to the lack of an integrated platform that combines real-time editing, communication, AI support, and session management. Current IDEs force developers to switch between multiple tools for code sharing, messaging, debugging, and version tracking, leading to inefficiency, errors, and version conflicts. To overcome these challenges, there is a need for a web-based Smart Collaborative IDE that unifies real-time coding, built-in chat, AI-powered debugging, session management, and contribution analytics to enhance productivity, accountability, and teamwork in software development.

Scope

- Enables real-time collaborative coding with host-controlled permissions, allowing multiple developers to work together in a synchronized environment.
- Provides AI-driven assistance for code correction, debugging, and optimization, reducing development errors and improving efficiency.
- Integrates a built-in chat system to support seamless communication and coordination without external tools.
- Offers AI-based contribution tracking to monitor individual performance, ensuring transparency and balanced workload distribution.
- Maintains a single downloadable file workspace, simplifying session management and eliminating complex folder or repository setups.
- Provides interactive dashboards for hosts with login, access control, real-time session monitoring, and contribution analytics.

Technological Stack

Front-end:

- HTML
- CSS
- Python
- Monaco

Back-end:

- Flask
- SocketHub

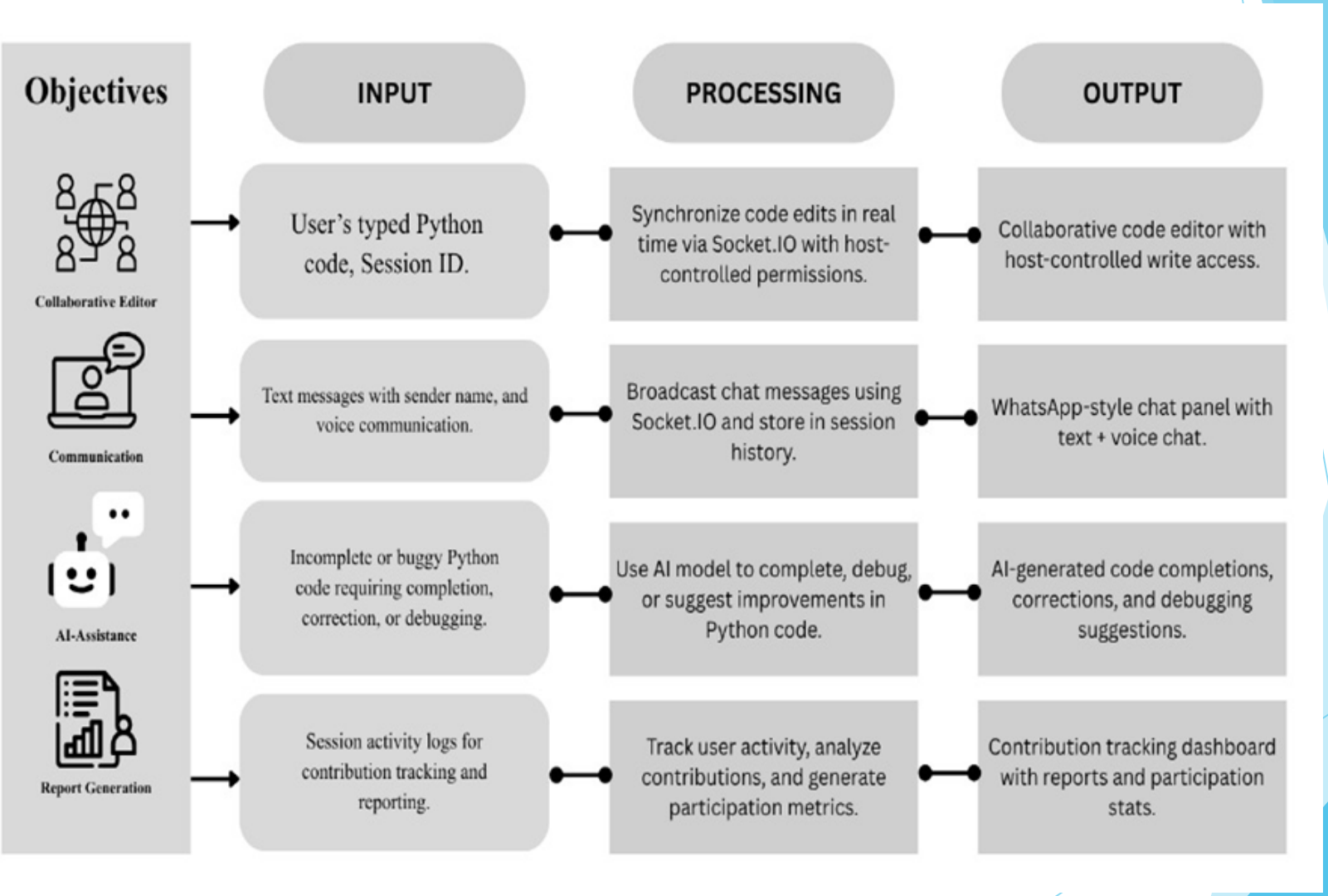
Storage/Cache :

- MySQL

Collabrative :

- Socketio
- HTTP/REST

Proposed system architecture



Prototype Design Demonstration

Not Applicable for Review 1

Implementation Status

Review Suggestions (Given in Last meeting)

Not Applicable for Review 1

References

1. Htoo Htoo Sandi Kyaw, Keiichi Kaneko, Nobuo Fuanbiki, "A Proposal of the Gamification of Programming Learning Assistant System," 2024 IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia), 2024 [1].
2. Haibin Lai, Peng Peng, Yuhui Lv, Peiwei Cai, Qiongxiong Ma, Zhun Zhang, "The Development of a Collaborative Learning Tool for AI Education: Corobolab," 2025 14th International Conference on Educational and Information Technology (ICEIT), 2025[2].
3. Saptaparni Kumar, Haochen Pan, Roger Wang, Lewis Tseng, "LiteDoc: Make Collaborative Editing Fast, Scalable, and Robust," 2020 [3].
4. Junbang Ma, "AI-Driven Web Application Automated Testing and Code Optimization," 2025 IEEE 6th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT), 2025 [4].
5. Alexey Svyatkovskiy, Sebastian Lee, Anna Hadjitofi, Maik Riechert, Juliana Vicente Franco, Miltiadis Allamanis, "Fast and Memory-Efficient Neural Code Completion," 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), 2021 [5].

6. Abdul Mannan Omi, Monir Hossain, Md Nahidul Islam, Tanni Mittra, "Multiple Authors Identification from Source Code Using Deep Learning Model," 2021 International Conference on Electronics, Communications and Information Technology (ICECIT), 2021[6].
7. Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, Gabriele Bavota, "An Empirical Study on the Usage of Transformer Models for Code Completion," IEEE Transactions on Software Engineering, 2021[7].

Thank You...!!